Université Clermont Auvergne

École Universitaire de Physique et d'Ingénierie

An AprilTag-Based Approach to Trajectory Recording and Relocalization for Mobile Robots

Second-year Master's project

Automation, Robotics track: Artificial Perception and Robotics

Authors:
Joao Pedro Martins do Lago Reis
Yann Kelvem Da Silva Ramos

Supervisor:
Dr. Sébastien Lengagne

Defense Date:
March 02, 2026

Aubière, Clermont-Ferrand

An AprilTag-Based Approach to Trajectory Recording and Relocalization for Mobile Robots

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

1 Introduction

Mobile robots operating in indoor laboratory environments cannot rely on satellite-based positioning systems and must instead estimate their pose using locally defined spatial references. Reliable localization therefore remains a fundamental requirement for controlled experimentation and autonomous operation, particularly in structured environments where repeatability and spatial consistency are expected **ULLAH2024100651**. Within this context, the objective of this project is to design a two-phase framework in which a mobile robot first records a reference trajectory during an initial run and subsequently estimates its pose with respect to that same reference. The purpose of this relocalization stage is to provide the geometric quantities required for the later implementation of a trajectory tracking control law. The scope of the work is thus not the design of the controller itself, but the establishment of a consistent and reproducible spatial framework that enables such control strategies to be developed and evaluated.

Although a trajectory can be recorded during an initial traversal, this action alone does not ensure that the same spatial path can be consistently referenced at a later time. In practice, odometry based motion estimation accumulates error over time due to sensor noise and unmodeled effects such as wheel slip, which leads to drift even in repeated runs **elhoushi2019odometry_survey**. Consequently, successive executions of the same nominal motion may produce trajectories that are not directly comparable in a common frame. To mitigate this, the recorded motion must be associated with stable spatial references so that the robot can reconstruct a consistent relationship between its current pose and the previously recorded path, which is a recurring requirement in mobile robot localization pipelines **ULLAH2024100651**. The technical problem therefore lies in defining a trajectory representation that remains coherent over time and in establishing a method to estimate the robot's pose relative to that reference during subsequent runs.

Based on these requirements, this work implements a two-phase system with an offline reconstruction stage and a relocalization stage designed for both offline analysis and online execution. Data collected during the initial traversal are processed offline to reconstruct the reference trajectory and to estimate an optimized marker map, since this step involves iterative estimation procedures with higher computational cost. During subsequent runs, relocalization relies on marker detection and pose estimation to compute the robot pose with respect to the reconstructed reference, which makes online deployment feasible.

The system was developed in ROS 2 Humble and validated on the AgileX LIMO platform. AprilTag v3 was selected based on recent evaluations reporting improved detection performance in practical conditions, and the tag36h11 family was used due to its widely adopted coding scheme with a large identifier set and error tolerance properties **olson2011apriltag, krogius2019flexible**. As a result, the system provides the reference trajectory and the relocalization outputs needed for trajectory tracking, and it is designed to be coupled with a controller developed as part of subsequent work.

2 System Requirements

2.1 Operational Context

The system is intended for controlled experimental environments in which AprilTag fiducial markers are deployed in advance. In this setting, the robot does not rely on satellite-based positioning and instead uses onboard perception together with a predefined marker reference to support trajectory recording and subsequent relocalization.

Two workflows are provided to support development and validation:

- a real-robot workflow validated on the AgileX LIMO platform,
- a simulation workflow used for repeatable testing and dataset generation.

The implementation is developed in ROS 2 Humble and organized as a modular workspace. The requirements listed in the following subsections specify the expected system functions, delivery constraints, and operational limitations.

Table 2.0: Functional Requirements

ID	Requirement
FR1	The system shall provide workflows for recording robot motion and perception data into rosbag2 datasets for offline processing.
FR2	The system shall process recorded rosbag2 datasets offline to generate a reference trajectory and a spatial map of detected AprilTags.
FR3	The offline processing stage shall export structured output files, including trajectory CSV and tag map YAML/CSV formats.
FR4	The system shall provide an online relocalization node capable of loading a reference trajectory, subscribing to a live pose topic, computing nearest-point distance and heading deviation, and publishing these quantities for controller integration.
FR5	The system shall publish visualization-compatible ROS topics enabling RViz display of reference trajectory, current pose, nearest reference pose, and relocalization error indicators.
FR6	The system shall provide launch files and profile scripts supporting both real and simulation workflows.
FR7	The system shall include documented execution procedures ensuring reproducibility from recorded datasets.

Table 2.1: Non-Functional Requirements

ID	Requirement
NFR1	Offline reconstruction shall be executable independently of live robot operation.
NFR2	Online relocalization shall operate within computational limits compatible with real-time ROS 2 execution.
NFR3	The architecture shall maintain separation between perception, offline reconstruction, and online relocalization components.
NFR4	The system shall ensure consistency of coordinate frames within the ROS 2 TF structure.
NFR5	Dataset files shall remain external to version control to maintain repository portability.

Table 2.2: Project Deliverables and Constraints

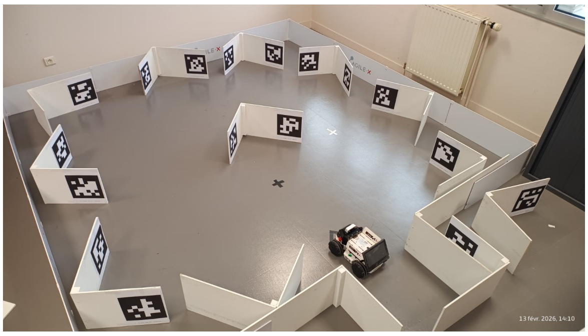
ID	Requirement
PR1	The project shall be delivered as a version-controlled ROS 2 workspace repository containing source code, launch files, scripts, and documentation.
PR2	The repository shall include structured workflows for both real and simulation execution.
C1	The system shall be implemented in ROS 2 Humble.
C2	The perception pipeline shall use AprilTag v3 with the tag36h11 family.
C3	Experimental validation shall be conducted on the AgileX LIMO platform.
C4	The system shall not include full autonomous navigation, SLAM of unknown environments, obstacle avoidance, or mission-level planning.

Table 2.3: Technical Limitations

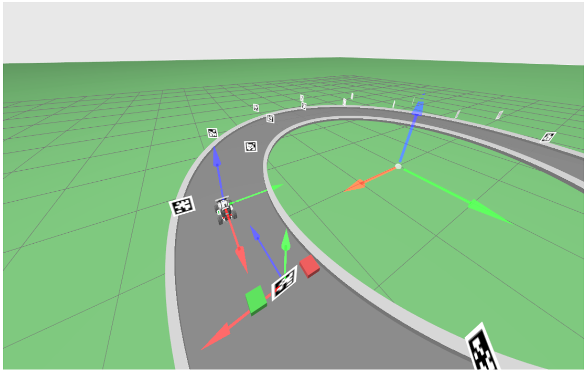
ID	Limitation
TL1	Accurate and stable pose estimation assumes that at least two AprilTags are simultaneously visible in the camera field of view. While pose can be computed from a single detection, multi-tag observations improve numerical stability and reduce sensitivity to noise and calibration errors.
TL2	Localization precision depends on camera calibration quality, tag size, distance to the tags, and image resolution. The system does not guarantee fixed absolute accuracy under arbitrary environmental conditions.

3 System Architecture

The system is implemented on the AgileX LIMO wheeled platform and uses a calibrated monocular RGB camera to observe AprilTags placed at fixed locations in the workspace and the Figures 3.1 illustrate the system in simulation and its corresponding real-world deployment. An initial traversal is recorded as a rosbag2 dataset so that the same sensor stream and time-stamped measurements can be processed offline to build a geometric reference.



(a) Real-world experimental setup with AgileX LIMO and fixed AprilTags.

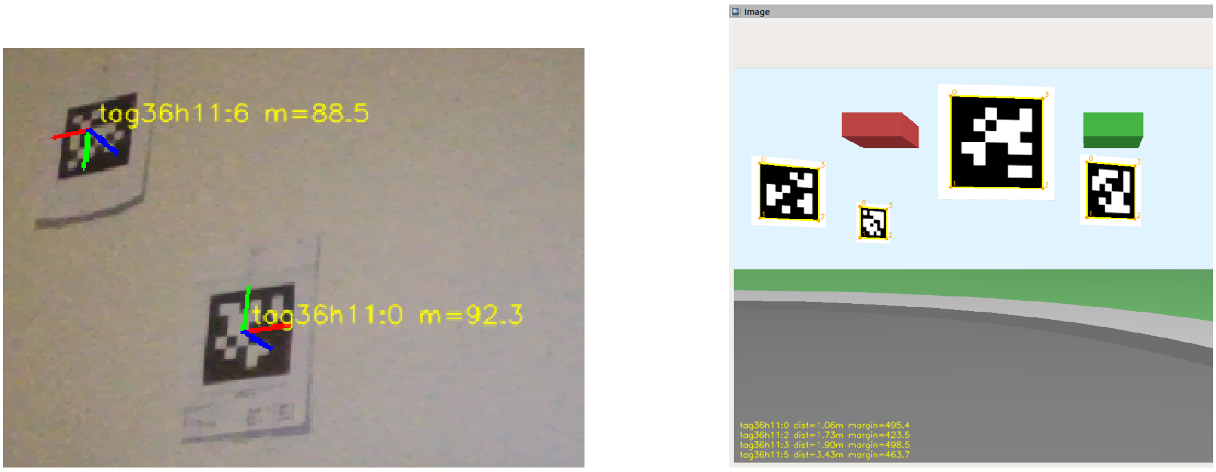


(b) Simulation environment used for dataset generation and validation.

Figure 3.1. Physical and simulated setups used to evaluate the proposed architecture.

The workflow is exercised in two settings, namely a real-robot setup and a simulation setup that reproduces the same sensor configuration and marker arrangement. Because the camera is rigidly mounted on the robot and the tags are treated as static landmarks in both settings, the same processing steps apply without changing the underlying formulation. The architecture is therefore presented as two consecutive phases. In the recording phase, the recorded traversal is processed to estimate an optimized tag map and a reference trajectory expressed consistently with that map. In the relocalization phase, new pose estimates are compared to the reference in order to compute the distance to the trajectory and the heading deviation relative to the local path direction, which are published as outputs for a trajectory tracking controller implemented outside the scope of this work.

Figure 3.2 illustrates representative camera frames acquired during both real and simulated executions. Detected markers are outlined and identified, and their estimated poses are expressed with respect to the camera frame. These per-frame relative transformations constitute the primary measurements used in the recording phase. Although each detection provides a six-degree-of-freedom estimate, the resulting poses are inherently local and subject to noise. As a result, consistent global structure must be recovered by integrating observations across multiple viewpoints.



(a) Example of AprilTag detection in the real environment.

(b) AprilTag detection in the simulated environment.

Figure 3.2. Representative detection, each detected tag yields a camera-relative pose estimate obtained through a PnP formulation.

3.1 Recording Phase

The recording phase processes the data collected during the initial traversal in order to construct a reference tag map and a corresponding camera trajectory. Figure 3.3 summarizes the computational steps involved. Each block in the pipeline corresponds to a specific estimation stage, which is detailed in the following subsections.

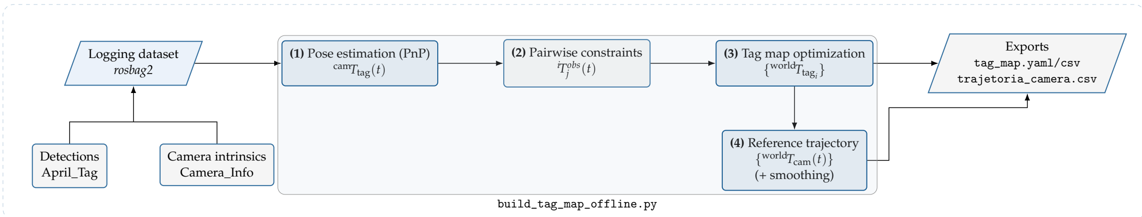


Figure 3.3. Architecture of the offline recording pipeline.

As indicated in block **(1)** of Figure 3.3, the first step of the recording phase estimates camera-to-tag relative poses by solving a Perspective-n-Point problem under calibrated intrinsics **Hartley2004** using the logged detections.

To construct a globally consistent representation of the environment, the local camera-to-tag measurements must be expressed in a common reference frame. In this work, rigid-body transformations are denoted by ${}^{A}T_{B} \in SE(3)$, representing the pose of frame B with respect to frame A **barfoot2017state**. Each detection provides a relative transformation ${}^{\text{cam}}T_{\text{tag}_i}(t)$ at time t. Since the camera pose varies along the traversal, each measurement relates a tag pose to a different camera coordinate system. If the pose of a marker tag_0 in the world frame is known or fixed, the camera pose follows from the inverse of the observed camera-to-tag transformation, as written in :

$$ {}^{\text{world}}T_{\text{cam}}(t) = {}^{\text{world}}T_{\text{tag}_0} \left({}^{\text{cam}}T_{\text{tag}_0}(t) \right)^{-1} \tag{3.1} $$

Equation (3.1) highlights both the utility and the limitation of single-frame detections. This map provides the global geometric reference required to express camera poses consistently across time. In this context, the tag map is defined as the set of world-referenced poses $\{{}^{\text{world}}T_{\text{tag}_i}\}$ estimated from all pairwise relative constraints accumulated along the traversal when an anchored tag is visible, a world-referenced camera pose estimate follows immediately. In practice, however, most tags do not have a pre-defined world pose. The recording phase therefore estimates the set of tag poses $\{{}^{\text{world}}T_{\text{tag}_i}\}$ that best explains the collection of camera-to-tag observations acquired along the traversal.

As illustrated in block **(2)**, frames containing multiple visible tags allow the construction of pairwise constraints. When two tags i and j are simultaneously detected, their relative transformation can be computed from the camera-centered estimates. Using the composition and inversion properties of rigid transformations **Hartley2004**, **barfoot2017state**, the observed relation is given by :

$$ {}^{i}T_j^{obs}(t) = \left({}^{\text{cam}}T_{\text{tag}_i}(t)\right)^{-1} {}^{\text{cam}}T_{\text{tag}_j}(t) \tag{3.2} $$

Each observation in Eq. (3.2) induces a constraint between the unknown world poses ${}^{\text{world}}T_{\text{tag}_i}$ and ${}^{\text{world}}T_{\text{tag}_j}$. In an ideal noise-free setting, the observed relative pose would match the relative pose implied by the world estimates :

$$ {}^{i}T_j^{pred} = \left({}^{\text{world}}T_{\text{tag}_i}\right)^{-1} {}^{\text{world}}T_{\text{tag}_j} \tag{3.3} $$

In practice, detection uncertainty, calibration inaccuracies, and viewing geometry variations lead to inconsistencies across multiple measurements of the same tag pair. Collecting all pairwise constraints along the traversal yields a network of relative pose relations between tags.

As shown in block **(3)**, estimating a consistent global map from such constraints is posed as a nonlinear least-squares problem, following the general principles used in geometric least-squares estimation and graph-based formulations **triggs1999ba**, **grisetti2010tutorial**. Let $\mathcal{E}$ denote the set of observed tag pairs. The tag poses are obtained by minimizing the sum of residuals over $\mathcal{E}$, as stated in :

$$ \min_{\{{}^{\text{world}}T_{\text{tag}_i}\}} \sum_{(i,j)\in\mathcal{E}} \left\|\text{err}_{ij}\right\|^2 \tag{3.4} $$

In Eq. (3.4), err_{ij} quantifies the discrepancy between the observed relative transformation in Eq. (3.2) and the relative transformation predicted by the current map estimate in Eq. (3.3). In practice, this discrepancy is decomposed into translational and rotational components.

To make the residual in Eq. (3.4) explicit, the predicted and observed relative transformations are written as ${}^{i}T_j^{pred} = (R_{ij}^{pred}, t_{ij}^{pred})$ and ${}^{i}T_j^{obs} = (R_{ij}^{obs}, t_{ij}^{obs})$. The translational component is defined as :

$$ r_{t,ij} = t_{ij}^{pred} - t_{ij}^{obs} \tag{3.5} $$

For the rotational component, the relative rotation between prediction and observation is computed by composition,

$$ R_{err,ij} = \left(R_{ij}^{pred}\right)^{\top} R_{ij}^{obs} \tag{3.6} $$

The matrix $R_{err,ij}$ is then mapped to a minimal three-parameter representation $r_{R,ij} \in \mathbb{R}^3$ using an axis–angle parametrization, which is consistent with the pose representation adopted for optimization **barfoot2017state**. The complete residual used in Eq. (3.4) is formed by stacking the translational and rotational components, $\mathrm{err}_{ij} = [\, r_{t,ij}^{\top} \;\; r_{R,ij}^{\top} \,]^{\top}$.

Each tag pose ${}^{\mathrm{world}}T_{\mathrm{tag}_i}$ is parameterized using a minimal six-dimensional representation consisting of a three-parameter rotation vector and a three-dimensional translation. The optimization variables therefore correspond to the stacked pose parameters of all non-anchored tags.

The minimization in Eq. (3.4) can be generalized to include a robust cost function,

$$\min_{\{{}^{\mathrm{world}}T_{\mathrm{tag}_i}\}} \sum_{(i,j)\in\mathcal{E}} \rho\left(\|\mathrm{err}_{ij}\|^2\right) \tag{3.7}$$

where $\rho(\cdot)$ denotes a robust loss function. In contrast to a purely quadratic penalty, robust losses reduce the influence of large residuals that may arise from occasional misdetections, partial occlusions, or inaccurate corner localization. Such formulations belong to the class of M-estimators commonly used in robust least-squares estimation **huber1964robust**.

In this implementation, a soft-ℓ_1 loss is employed, this loss behaves quadratically for small residuals, preserving the local convergence properties of standard least squares, and transitions to approximately linear growth for larger errors. As a result, residuals with large magnitude contribute less aggressively to the total cost, preventing them from dominating the optimization process. This choice is consistent with the presence of occasional inconsistent tag observations while maintaining sensitivity to well-conditioned measurements.

The solution is obtained iteratively by updating the pose parameters so as to decrease the total residual error. One tag pose is held fixed throughout the optimization in order to eliminate the global gauge freedom discussed previously. The nonlinear least-squares problem is solved using an iterative trust-region method, as implemented in standard numerical optimization libraries. At each iteration, the pose parameters are updated so as to reduce the total robust cost defined in Eq. (3.7). This procedure follows the classical Gauss–Newton framework for nonlinear least squares, adapted to the chosen minimal pose parametrization **triggs1999ba**.

Finally, as indicated in block **(4)**, the optimized tag map is used to reconstruct the camera trajectory. For each observation of a tag i at time t, a world-referenced camera pose estimate is obtained according to:

$${}^{\mathrm{world}}T_{\mathrm{cam}}^{(i)}(t) = {}^{\mathrm{world}}T_{\mathrm{tag}_i} \left({}^{\mathrm{cam}}T_{\mathrm{tag}_i}(t)\right)^{-1} \tag{3.8}$$

When multiple tags are detected in the same frame, several estimates $\{{}^{\mathrm{world}}T_{\mathrm{cam}}^{(i)}(t)\}$ are obtained. In the absence of noise, these estimates would coincide, in practice, small discrepancies arise due to measurement uncertainty and residual map errors. A single pose per frame is obtained either by averaging the rigid transformations, by selecting the estimate with the smallest reprojection residual, or by solving a local nonlinear least-squares problem that minimizes the residual error between predicted and observed camera-to-tag transformations.

The resulting sequence $\{{}^{\mathrm{world}}T_{\mathrm{cam}}(t)\}$ defines the reference trajectory recorded during the initial traversal. Since frame-wise estimation may introduce high-frequency variations, the trajectory is optionally regularized in time prior to export.

The temporal regularization is applied directly to the discrete sequence $\{{}^{\mathrm{world}}T_{\mathrm{cam}}(t)\}$ and penalizes large pose differences between consecutive timestamps while preserving the spatial structure imposed by the optimized tag map. This operation improves temporal consistency without altering the underlying map estimation. The final smoothed trajectory constitutes the geometric reference used in the subsequent *relocalization phase*, where live pose estimates are compared against this stored reference in order to compute distance and heading deviations.

3.2 Relocalization Phase

The relocalization phase evaluates the geometric deviation between an estimated camera pose and the optimized reference trajectory obtained in the recording phase. In this stage, both the tag map $\{{}^{world}T_{tag_i}\}$ and the reference trajectory $\{{}^{world}T_{cam}(t_k)\}$ are assumed to be fixed and previously computed. Figure 3.4 summarizes the computational structure adopted for relocalization, separating offline analysis from online execution.

In contrast to the recording phase, no map optimization or trajectory reconstruction is performed here. Instead, the system receives a query pose ${}^{world}T^{q}_{cam} \in SE(3)$ and computes its geometric relationship with respect to the discrete reference trajectory. The resulting quantities are a distance-to-trajectory metric and a heading deviation, which are defined formally in the following subsection.

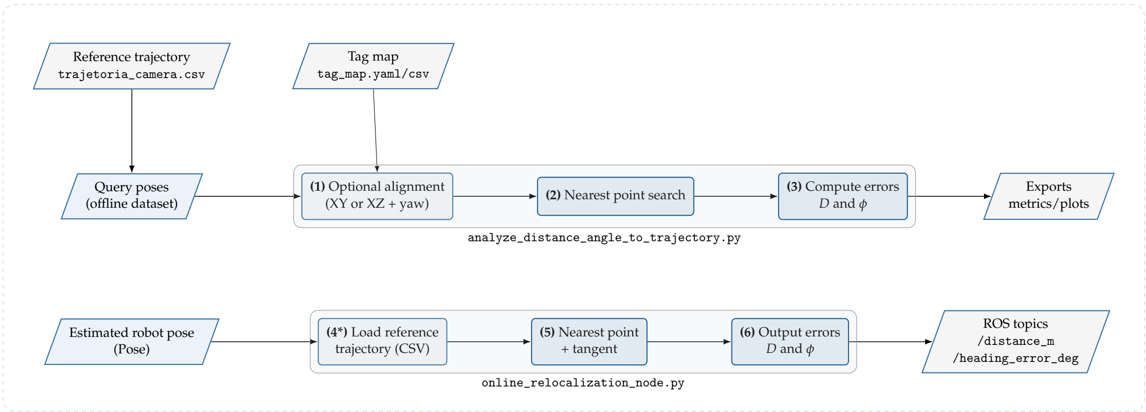


Figure 3.4. Architecture of the relocalization pipeline with separated offline analysis and online execution.

3.2.1 Offline evaluation

As presented in block **(1)**, the first operation in the relocalization workflow optionally performs a planar alignment between the query dataset and the stored reference trajectory. This step compensates for possible coordinate discrepancies arising from independent reconstructions or scale differences during map generation.

Let $\{p_k^{ref}\} \subset \mathbb{R}^2$ denote the planar positions extracted from the reference trajectory $\{{}^{world}T_{cam}(t_k)\}$ and let $\{p_\ell^{q}\} \subset \mathbb{R}^2$ denote the planar positions of the query dataset. When a set of common tags is available in both datasets, a transformation is estimated so that the transformed query positions best match the reference positions in a least-squares sense.

In the most general case considered here, the alignment is modeled as a planar similarity transformation,

$$p_\ell^{q,aligned} = sRp_\ell^{q} + t \tag{3.9}$$

where $R \in SO(2)$ represents a planar rotation, $t \in \mathbb{R}^2$ is a translation vector, and $s \in \mathbb{R}^+$ is a uniform scale factor. When scale correction is not required, the model reduces to a rigid transformation with $s = 1$.

The optimal parameters (R, t, s) are obtained by minimizing the squared alignment residual over the set of matched tag correspondences:

$$\min_{R,t,s} \sum_{m \in \mathcal{C}} \left\| p_m^{ref} - \left(sRp_m^q + t \right) \right\|^2 \tag{3.10}$$

where $\mathcal{C}$ denotes the set of tag identifiers observed in both datasets. Problems of this form admit closed-form solutions based on singular value decomposition and correspond to the classical orthogonal Procrustes formulation **Schoenemann1966**.

In practice, inconsistencies may arise due to partial overlap or incorrect detections. To reduce the influence of such outliers, a RANSAC scheme **Fischler1981** is applied prior to the final least-squares estimation. The transformation computed from inlier correspondences is then applied to all query poses. If no valid correspondences are available, or if alignment is explicitly disabled, the query poses are evaluated in their original coordinate frame. Once both trajectories are expressed in a common planar reference, the relocalization task reduces to determining the geometric association between each query pose and the reference trajectory.

As illustrated in block **(2)**, once the query and reference trajectories are expressed in a common planar frame, the relocalization problem reduces to establishing a geometric association between a query pose and the discrete reference trajectory.

Let the reference trajectory exported in the recording phase be represented by the set:

$$\mathcal{T}_{ref} = \left\{ {}^{world}T_{cam}(t_k) \right\}_{k=1}^{N} \tag{3.11}$$

and let $p_k^{ref} \in \mathbb{R}^2$ denote the planar position extracted from each pose. Given a query pose ${}^{world}T_{cam}^{q}$ with planar position $p^q \in \mathbb{R}^2$, the nearest reference association is defined as:

$$k^* = \arg \min_{k \in \{1,\dots,N\}} \left\| p^q - p_k^{ref} \right\|_2 \tag{3.12}$$

Equation (3.12) defines the reference sample whose planar position is closest to the query position in the Euclidean sense. The selected index k^* serves as the geometric anchor for evaluating the deviation between the live pose and the stored reference trajectory.Based on this association, the relocalization phase proceeds to quantify the positional and angular discrepancy with respect to the selected reference pose.

As presented in block **(3)**, once the nearest association index k^* is available, the relocalization pipeline evaluates two geometric error metrics defined with respect to the associated reference sample.

The positional error is defined as the Euclidean distance between the query planar position $p^q \in \mathbb{R}^2$ and the associated reference planar position $p_{k^*}^{ref} \in \mathbb{R}^2$:

$$D = \left\| p^q - p_{k^*}^{ref} \right\|_2 \tag{3.13}$$

To compute an angular error, a reference direction is defined locally along the discrete reference trajectory. Let $p_k^{ref} = (x_k^{ref}, z_k^{ref})$ denote the planar reference samples. The reference heading at index k is obtained from the local tangent using a centered difference:

$$\theta_k^{ref} = \operatorname{atan2}\Big(z_{k+1}^{ref} - z_{k-1}^{ref},\ x_{k+1}^{ref} - x_{k-1}^{ref} \Big) \tag{3.14}$$

with a one-sided variant at the endpoints. Let θ^q denote the query planar heading extracted from the query pose orientation in the same plane. The heading error is then defined as:

$$\phi = \mathrm{wrap}_{(-\pi,\pi]}\left(\theta^q - \theta^{ref}_{k^*}\right) \tag{3.15}$$

where $\mathrm{wrap}_{(-\pi,\pi]}(\cdot)$ maps the angular difference to the principal interval.

Equations (3.13)–(3.15) specify the outputs computed at this stage for each evaluated query pose. These values are then forwarded to the reporting stage in the offline workflow.

As presented in block **(3)** of the offline branch, the computed values of D and ϕ are exported for quantitative assessment. Given a sequence of query poses $\{{}^{world}T^q_{cam}(t_\ell)\}$, Equations (3.13) and (3.15) are evaluated for each timestamp. This produces two discrete time series:

$$\{D(t_\ell)\}_{\ell=1}^{M} \qquad \{\phi(t_\ell)\}_{\ell=1}^{M} \tag{3.16}$$

where M denotes the number of evaluated query samples.

These sequences are stored together with the associated timestamps and reference indices, enabling post-processing and statistical evaluation. Summary quantities such as mean, median, percentile levels, and maximum deviation are derived directly from the exported series. Additionally, planar visualizations are generated to illustrate the spatial relationship between the query samples and the reference trajectory. This offline reporting stage provides a quantitative characterization of relocalization performance over an entire traversal.

3.2.2 Online execution

As depicted in block **(4*)**, the online branch starts by loading the reference trajectory generated in the recording phase. The file `trajetoria_camera.csv` provides the discrete sequence $\mathcal{T}_{ref} = \{{}^{world}T_{cam}(t_k)\}_{k=1}^{N}$.

In online execution, $\mathcal{T}_{ref}$ is treated as fixed data. The node parses the file once and stores the planar reference samples $\{p^{ref}_k\}_{k=1}^{N}$ together with their associated local headings $\{\theta^{ref}_k\}_{k=1}^{N}$, computed from the discrete trajectory tangent as in Eq. (3.14). This initialization establishes the reference set required to evaluate incoming pose estimates without modifying the stored trajectory. Once the reference trajectory is available in memory, each incoming pose is processed by evaluating the nearest-neighbor association in Eq. (3.12) and subsequently computing D and ϕ.

As presented in block **(5)**, the online node performs a nearest-sample association between the current pose estimate and the fixed reference set. At each time t, the node receives a query pose ${}^{world}T^q_{cam}(t)$ and extracts its planar position $p^q(t)$. The association index is then obtained by directly applying the nearest-neighbor criterion in Eq. (3.12), yielding:

$$k^*(t) = \arg\min_{k\in\{1,\ldots,N\}} \left\| p^q(t) - p^{ref}_k \right\|_2 \tag{3.17}$$

The reference sample indexed by $k^*(t)$ provides the corresponding stored heading $\theta^{ref}_{k^*(t)}$, which anchors the frame-by-frame evaluation of the geometric errors defined in Eqs. (3.13) and (3.15). The resulting association $\left(p^q(t), p^{ref}_{k^*(t)}\right)$ is therefore the immediate input to the error output stage described next.

As summarized in block **(6)**, the online execution evaluates the geometric errors for each incoming pose and makes them available as real-time outputs.

Given the association index $k^*(t)$, the positional and angular deviations are computed. These quantities are evaluated independently at each time step and therefore define two time-varying signals, $D(t)$ and $\phi(t)$, describing the instantaneous geometric discrepancy between the current pose estimate and the stored reference trajectory.

The online node publishes these values as scalar outputs, together with visualization elements that indicate the current pose, the associated reference sample, and the geometric connection between them. This structure enables direct inspection of relocalization behavior during execution and provides the signals required for potential control integration.

The experimental section evaluates these outputs quantitatively by analyzing their temporal evolution under different traversal conditions, thereby assessing the consistency between live pose estimates and the reference trajectory established during the recording phase.

4 Results

4.1 Validation of the Map Optimization under Controlled Conditions

The algorithm validation was performed using synthetic datas, which allows for direct comparison with Ground Truth to quantify optimization precision and robustness.

Initially, the system was tested with perfect detections, free of drift. For this data, the residual error was null, confirming the geometric consistency of the algorithm. The metrics are presented in Table 4.1.

Metric	Mean Error	Minimum	Maximum
Camera Trajectory	0.0000 m/0.00°	0.0000 m/0.00°	0.0000 m/0.00°
Tag Poses	0.0000 m/0.00°	0.0000 m/0.00°	0.0000 m/0.00°

Table 4.1. Residual errors in the ideal scenario (no noise).

Figure 4.1 presents the visual results for this scenario. On the left, the alignment of estimated tags with the real scenario is observed, showing that the tags were perfectly aligned as suggested by the table, without errors. On the right, the optimized trajectory is superimposed on the reference circular trajectory, verifying that the trajectory was perfectly recovered without errors.

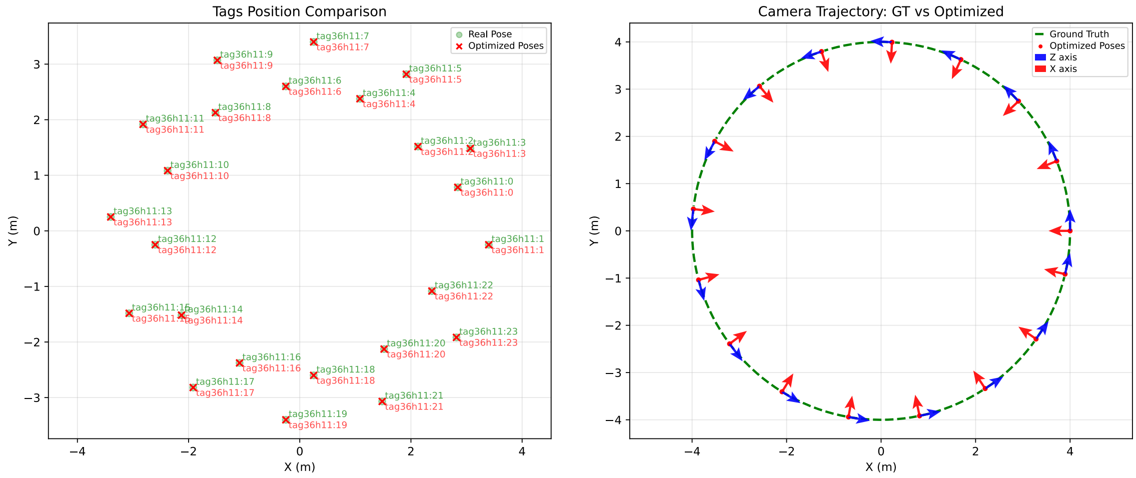


Figure 4.1. The tag map and the camera trajectory.

To evaluate robustness, Gaussian noise of 10 cm was injected into the position measurements. The optimization algorithm demonstrated high efficacy in filtering this noise, as shown in the data presented in Table 4.2.

Metric	**Mean Error**	**Minimum**	**Maximum**
Camera Trajectory	0.0584 m/0.97°	0.0126 m/0.11°	0.1485 m/2.67°
Tag Poses	0.0364 m/0.73°	0.0156 m/0.73°	0.0583 m/0.73°

Table 4.2. Error statistics under 10 cm Gaussian noise.

Figure 4.2 illustrates the visual comparison for this noisy scenario, demonstrating that the system can recover the map structure and trajectory with centimeter-level precision even under significant uncertainty. Even with 10 cm Gaussian noise, the algorithm proved robust: it reduced tag pose error from $\approx$ 10 cm to about 3 cm and camera pose error to roughly 6 cm. The visualization confirms that tags remain well localized and the trajectory is effectively recovered despite the small residual error.

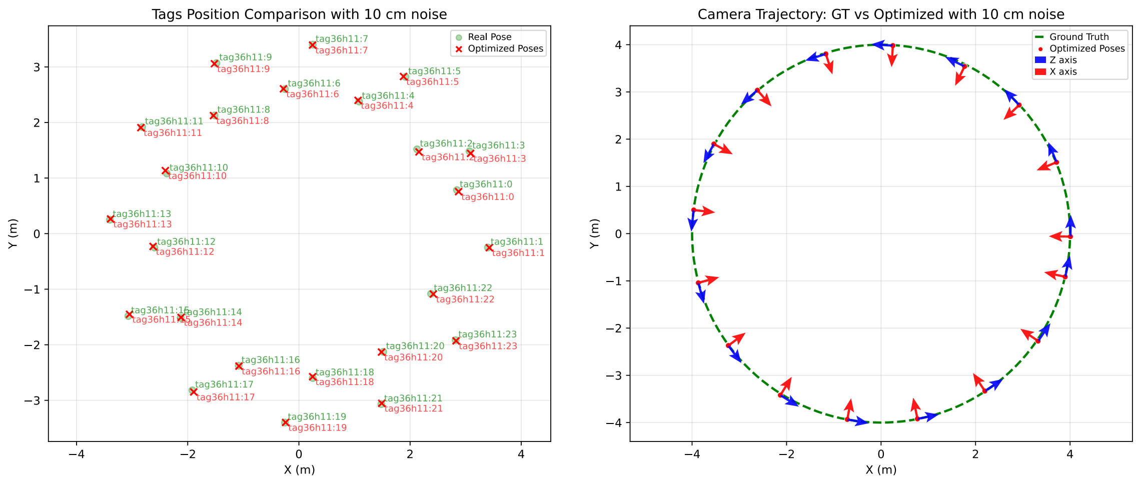


Figure 4.2. The map recovery and the trajectory, for noise measurements

The algorithm validation was performed using synthetic datas, which allows for direct comparison with Ground Truth to quantify optimization precision and robustness.

5 Conclusion